

COURSE CODE/TITLE: CPE 010-CPE21S1 - Data Structures and Algorithms	SECTION: CPE21S1
DATE PERFORMED: Sep 19, 2026	DATE SUBMITTED: Sep 21, 2026
NAME: Mendoza, Roy Gabriel I.	INSTRUCTOR: Engr. Jimlord Quejado

ACTIVITY NO. 9.1	Tree ADT
-------------------------	-----------------

1. PROCEDURE OUTPUT

What procedures did you perform?

ILO A: Create C++ code to implement a general tree, binary tree and binary search tree.		
T A S K 1:	CODE:	
	<table border="1"> <tr> <td>Header</td> <td></td> </tr> </table>	Header
Header		

```

1 //
2 // Created by royga on 9/19/2026.
3 //
4
5 #ifndef UNTITLED1_GENTREE_TASK1_H
6 #define UNTITLED1_GENTREE_TASK1_H
7
8 #include <iostream>
9 template <typename T>
10 class genTree {
11 private:
12     T key;
13     genTree<T>* firstChild;
14     genTree<T>* nextSibling;
15
16 public:
17     genTree(T value);
18     ~genTree();
19     genTree<T>* insert(T value);
20     void printTree(int depth = 0);
21 };
22
23 template <typename T>
24 genTree<T>::genTree(T value) {
25     key = value;
26     firstChild = nullptr;
27     nextSibling = nullptr;
28 }
29

```

```

30     template <typename T>
31     ↩ genTree<T>::~~genTree() {
32         delete firstChild;
33         delete nextSibling;
34     }
35
36     template <typename T>
37     ↩ genTree<T>* genTree<T>::insert(T value) {
38         genTree<T>* node = new genTree<T>(value);
39
40         if (firstChild == nullptr) {
41             firstChild = node;
42         } else {
43             genTree<T>* current = firstChild;
44             while (current->nextSibling != nullptr) {
45                 current = current->nextSibling;
46             }
47             current->nextSibling = node;
48         }
49         return node;
50     }
51
52     template <typename T>
53     ↩ void genTree<T>::printTree(int depth) {
54         for (int i = 0; i < depth; i++) {
55             std::cout << "  ";
56         }
57         std::cout << key << "\n";
58         for (genTree<T>* child = firstChild; child != nullptr; child = child->nextSibling) {
59             child->printTree(depth + 1);
60         }
61     }
62
63 #endif //UNTITLED1_GENTREE_TASK1_H

```

Main

```
1 //  
2 // Created by royga on 9/19/2026.  
3 //  
4 #include <iostream>  
5 #include "genTree.h"  
6  
7 int main() {  
8     genTree<char> tree(value: 'A');//ROOT  
9  
10    //CHILDS OF A  
11    tree.insert('B');  
12    tree.insert('C');  
13    genTree<char>* D = tree.insert('D');  
14    genTree<char>* E = tree.insert('E');  
15    genTree<char>* F = tree.insert('F');  
16    genTree<char>* G = tree.insert('G');  
17  
18    //grandchild of A  
19    D->insert('H');  
20    E->insert('I');  
21    genTree<char>* J = E->insert('J');  
22    F->insert('K');  
23    F->insert('L');  
24    F->insert('M');  
25    G->insert('N');  
26  
27    // Greatgrand of A  
28    J->insert('P');  
29    J->insert('Q');  
30  
31    tree.printTree();  
32  
33    return 0;  
34 }
```

T
A
S
K
2:

NODE	HEIGHT	DEPTH
A	3	0
B	0	1
C	0	1
D	1	1
E	2	1
F	1	1
G	1	1
H	0	2
I	0	2
J	1	2
K	0	2
L	0	2
M	0	2
N	0	2
P	0	3
Q	0	3

ILO B: Create C++ code for implementation of tree traversal methods such as pre-order, in-order and post-order traversal.

3.1

Pre-Order: A B C D H E I J P Q F K L M G N
 Post-Order: B C H D I P Q J E K L M F N G A
 In-order: B A C H D I E P Q K F L M N G

3.2

PRE-ORDER

```
125 // Pre-order
126 template <typename T>
127 bool genTree<T>::preOrder(T value) {
128     if (key == value) {
129         return true;
130     }
131     for (genTree<T>* child = firCld; child != nullptr; child = child->nxtSib) {
132         if (child->preOrder(value)) {
133             return true;
134         }
135     }
136     return false;
137 }
```

```
33 // TASK 3.2
34 std::cout << "\nPre-order: ";
35 tree.preOrder();

Pre-order:  A B C D H E I J P Q F K L M G N
```

POST-ORDER

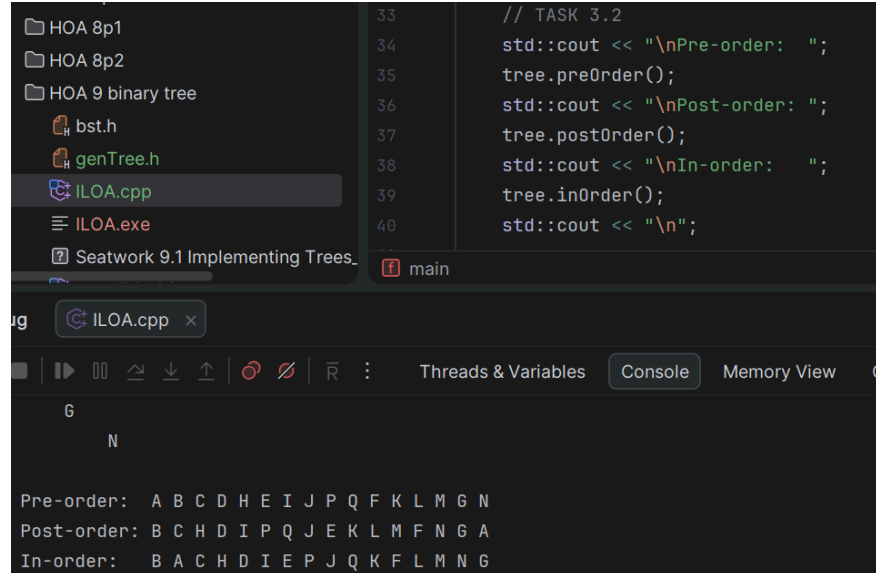
```
139 // Post-order
140 template <typename T>
141 bool genTree<T>::postOrder(T value) {
142     for (genTree<T>* child = firCld; child != nullptr; child = child->nxtSib) {
143         if (child->postOrder(value)) {
144             return true;
145         }
146     }
147     return key == value;
148 }
```

```
36 std::cout << "\nPost-order: ";
37 tree.postOrder();
38 std::cout << "\nIn-order: ";

Pre-order:  A B C D H E I J P Q F K L M G N
Post-order: B C H D I P Q J E K L M F N G A
```

IN-ORDER

```
150 // In-order
151 template <typename T>
152 bool genTree<T>::inOrder(T value) {
153     if (firCld == nullptr) {
154         return key == value;
155     }
156     if (firCld->inOrder(value)) {
157         return true;
158     }
159     if (key == value) {
160         return true;
161     }
162     for (genTree<T>* child = firCld->nxtSib; child != nullptr; child = child->nxtSib) {
163         if (child->inOrder(value)) {
164             return true;
165         }
166     }
167     return false;
168 }
```



The screenshot shows a C++ IDE with a file explorer on the left, source code in the center, and a console window at the bottom. The file explorer lists folders 'HOA 8p1', 'HOA 8p2', and 'HOA 9 binary tree', and files 'bst.h', 'genTree.h', 'ILOA.cpp', 'ILOA.exe', and 'Seatwork 9.1 Implementing Trees...'. The source code in the center shows the implementation of the in-order traversal function. The console window at the bottom displays the output of the program, showing the pre-order, post-order, and in-order traversals of a binary tree.

```
33 // TASK 3.2
34 std::cout << "\nPre-order: ";
35 tree.preOrder();
36 std::cout << "\nPost-order: ";
37 tree.postOrder();
38 std::cout << "\nIn-order: ";
39 tree.inOrder();
40 std::cout << "\n";
```

6
N

Pre-order: A B C D H E I J P Q F K L M G N
Post-order: B C H D I P Q J E K L M F N G A
In-order: B A C H D I E P J Q K F L M N G

OBSERVATION

From my observation of both the self made and the code output, they are the same, so what I did in the manual was right.

3.3

```
170     template <typename T>
171     → ↵ void genTree<T>::findData(int choice, T value) {
172         bool found = false;
173
174     ↵     switch (choice) {
175         |     case 1: found=preOrder(value);
176         |         break;
177         |     case 2: found=postOrder(value);
178         |         break;
179         |     case 3: found= inOrder(value);
180         |         break;
181         |     default:
182         |         return;
183     }
184     ↵     if (found) {
185         |         std::cout <<"["<< value << "]was found!" << "\n";
186     }
```

HOA 3p1

HOA 4 - stacks

HOA 5p1

HOA6p1

HOA 7p1

HOA 8p1

HOA 8p2

HOA 9 binary tree

bst.h

42

43

44

45

46

47

48

49

// TASK 3.3-find data: 1=pre::: 2=post;;; 3=in

std::cout << "\nfindData:\n";

tree.findData(choice:1, value: 'J');

tree.findData(choice:2, value: 'J');

tree.findData(choice:3, value: 'J');

tree.findData(choice:4, value: 'J');

tree.findData(choice:1, value: 'W');

main

g ILOA.cpp x

Threads & Variables

Console

Memory View

GDB

Live Watche

Post-order: B C H D I P Q J E K L M F N G A
In-order: B A C H D I E P J Q K F L M N G

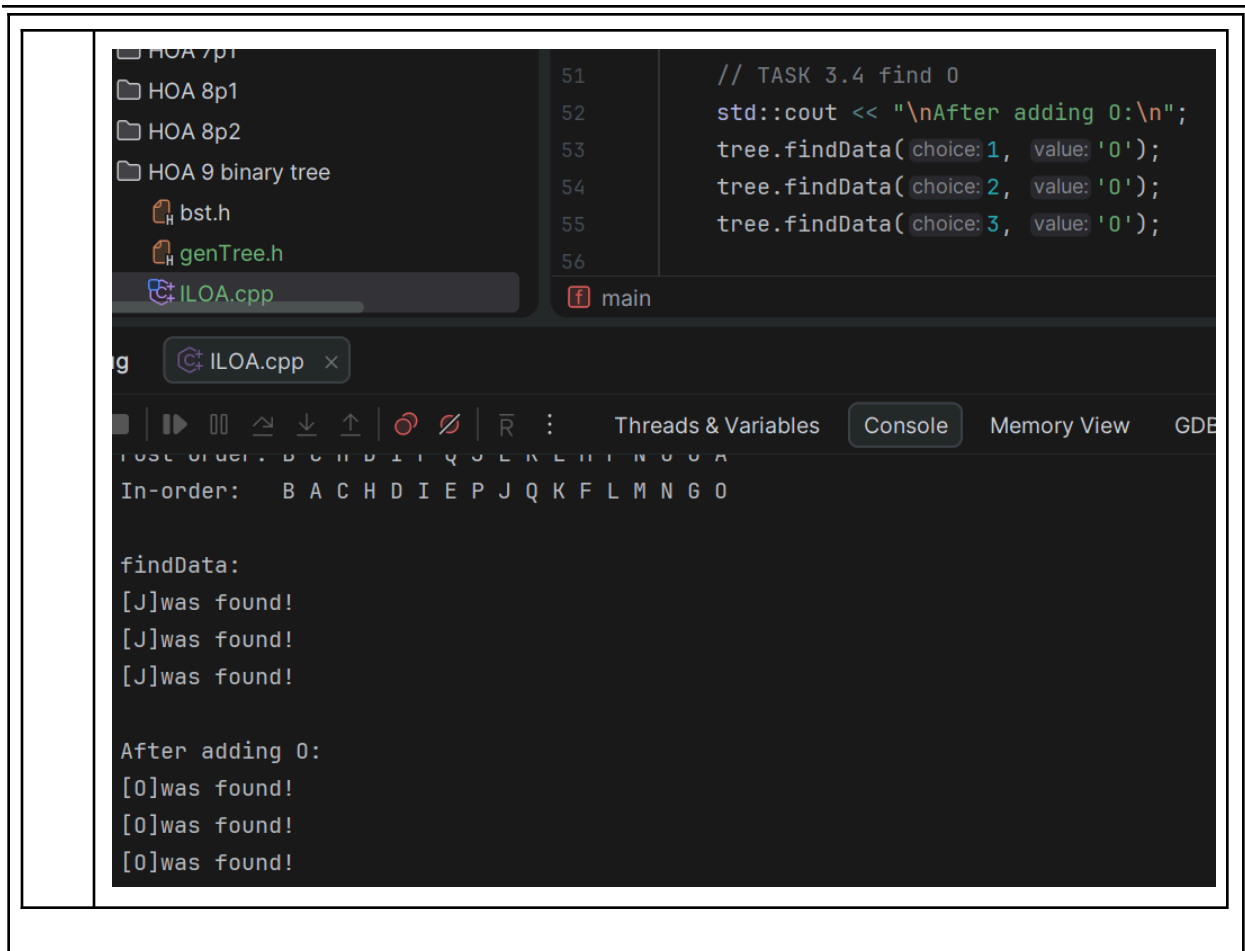
findData:
[J]was found!
[J]was found!
[J]was found!

Before adding 0:

After adding 0:
[0]was found!
[0]was found!
[0]was found!

3.4

```
27      // Greatgrand of A
28      J->insert('P');
29      J->insert('Q');
30      G->insert('O');
31
32      tree.printTree();
33
34      // TASK 3.2
35      std::cout << "\nPre-order: ";
36      tree.preOrder();
37      std::cout << "\nPost-order: ";
38      tree.postOrder();
39      std::cout << "\nIn-order: ";
40      tree.inOrder();
41      std::cout << "\n";
42
43      // TASK 3.3-find data: 1=pre::: 2=post;;; 3=in
44      std::cout << "\nfindData:\n";
45      tree.findData( choice:1, value: 'J');
46      tree.findData( choice:2, value: 'J');
47      tree.findData( choice:3, value: 'J');
48      tree.findData( choice:4, value: 'J');
49      tree.findData( choice:1, value: 'W');
```



2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

STEP 1:

HEAD
R

```
1 //
2 // Created by royga on 9/17/202
3 //
4
5 #ifndef UNTITLED1_BST_H
6 #define UNTITLED1_BST_H
7
8 #include <iostream>
9
10 template <typename T>
11 class baiSerTree {
12 private:
13     T key;
14     baiSerTree<T>* left;
15     baiSerTree<T>* right;
16
17 public:
18     // Constructor
19     ↩ ↪ baiSerTree(T value);
20
21     // Destructor
22     ↩ ↪ ~baiSerTree();
23
24     // Insert function
25     ↩ ↪ void insert(T value);
26
27     // Traversing algorithms
28     ↩ ↪ void preOrder();
```

```
33     template <typename T>
34     → ↵ baiSerTree<T>::baiSerTree(T value) {
35         key = value;
36         left = nullptr;
37         right = nullptr;
38     }
39
40     template <typename T>
41     → ↵ baiSerTree<T>::~~baiSerTree() {
42         delete left;
43         delete right;
44     }
45
46     template <typename T>
47     → ↵ void baiSerTree<T>::insert(T value) {
48         ↵     if (value < key) {
49             ↵         if (left == nullptr) {
50                 |         left = new baiSerTree<T>(value);
51             } else {
52                 |         left->insert(value);
53             }
54         } else if (value > key) {
55             ↵         if (right == nullptr) {
56                 |         right = new baiSerTree<T>(value);
57             } else {
58                 |         right->insert(value);
59             }
60         }
61     }
```

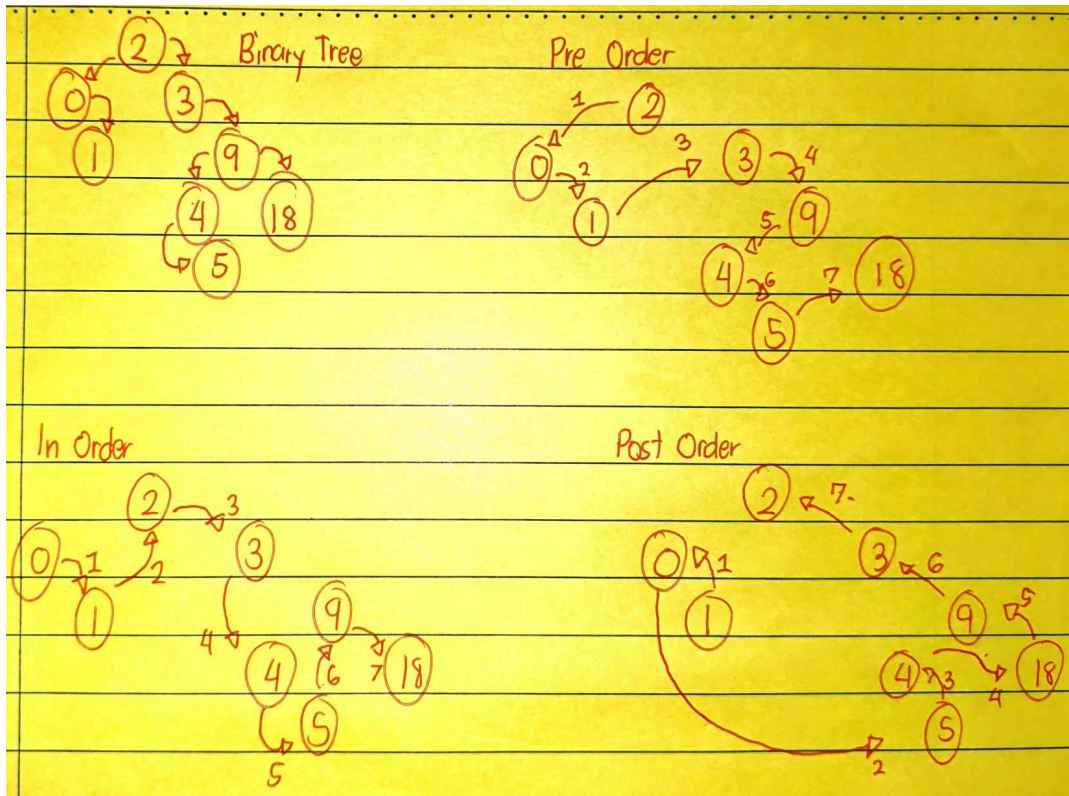
```
63 // Pre-order: Root -> Left -> Right
64 template <typename T>
65 → ↵ void baiSerTree<T>::preOrder() {
66     std::cout << key << " ";
67     ↵ if (left != nullptr) {
68         |     left->preOrder();
69     }
70     ↵ if (right != nullptr) {
71         |     right->preOrder();
72     }
73 }
74
75 // In-order: Left -> Root -> Right
76 template <typename T>
77 → ↵ void baiSerTree<T>::inOrder() {
78     ↵ if (left != nullptr) {
79         |     left->inOrder();
80     }
81     |     std::cout << key << " ";
82     ↵ if (right != nullptr) {
83         |     right->inOrder();
84     }
85 }
86
87 // Post-order: Left -> Right -> Root
88 template <typename T>
89 → ↵ void baiSerTree<T>::postOrder() {
90     ↵ if (left != nullptr) {
91         |     left->postOrder();
92     }
```

```
94         right->postOrder();
95     }
96     std::cout << key << " ";
97 }
98
99 #endif //UNTITLED1_BST_H
```

MAIN

```
1 //
2 // Created by royga on 9/21/2026.
3 //
4 #include <iostream>
5 #include "bst.h"
6
7 int main() {
8     //Step 1: Implement a binary search tree that will take the following values: 2, 3, 9, 18, 0, 1, 4, 5.
9     baiSerTree<int> tree(value:2);
10
11     tree.insert(value:3);
12     tree.insert(value:9);
13     tree.insert(value:18);
14     tree.insert(value:0);
15     tree.insert(value:1);
16     tree.insert(value:4);
17     tree.insert(value:5);
18
19     //Step 3: Compare the different traversal methods. In-order traversal was performed with what function?
20     std::cout << "In-order: ";
21     tree.inOrder();
22
23     std::cout << "\nPre-order: ";
24     tree.preOrder();
25
26     std::cout << "\nPost-order: ";
27     tree.postOrder();
28     std::cout << "\n";
29
30     return 0;
31 }
```

STEP 2:



STEP 3:

IN ORDER

```

75 // In-order: Left -> Root -> Right
76 template <typename T>
77 void baiSerTree<T>::inOrder() {
78     if (left != nullptr) {
79         left->inOrder();
80     }
81     std::cout << key << " ";
82     if (right != nullptr) {
83         right->inOrder();
84     }
85 }

```

	<pre> 18 19 //Step 3: Compare the different traversal methods. In-order traversal was performed with what function? 20 std::cout << "In-order: "; 21 tree.inOrder(); </pre> In-order: 0 1 2 3 4 5 9 18
PRE ORDER	<pre> 62 63 // Pre-order: Root -> Left -> Right 64 template <typename T> 65 → void baiSerTree<T>::preOrder() { 66 std::cout << key << " "; 67 if (left != nullptr) { 68 left->preOrder(); 69 } 70 if (right != nullptr) { 71 right->preOrder(); 72 } 73 } </pre> <pre> 22 23 std::cout << "\nPre-order: "; 24 tree.preOrder(); 25 </pre> Pre-order: 2 0 1 3 9 4 5 18

POST ORDER	<pre> 87 // Post-order: Left -> Right -> Root 88 template <typename T> 89 → void baiSerTree<T>::postOrder() { 90 if (left != nullptr) { 91 left->postOrder(); 92 } 93 if (right != nullptr) { 94 right->postOrder(); 95 } 96 std::cout << key << " "; 97 } 98 99 #endif //UNTITLED1_BST_H </pre> <pre> 26 std::cout << "\nPost-order: "; 27 tree.postOrder(); 28 std::cout << "\n"; </pre> Post-order: 1 0 5 4 18 9 3 2
OBSERVATION /	>Based on my observation, the one that I made in the paper and the code, it has the same result and the values are the same.
CONCLUSION	> Overall, this activity was able to make me understand the tree, although this is not a binary tree(because of the examples and task) but overall I get the idea of how it functions. The procedure was able to make me a code that I can use to basically all of the tree functions that are needed. And in the supplementary, I was able to just use the code(header) in the procedure, and not change anything, since it is already done there. Overall, I can properly say that I actually did well here, just the post order is the one that gave me a headache to do, but thanks to online resources, I was able to do it properly, and with the help of friend collaboration we were able to finish this right(with fighting on how to do it).

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: None

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: None

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: None

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.



Please have the honor of pledge in the comment section or at the bottom of your activity document/exam or any form of file submission.



I affirm that I will not give or receive any unauthorized help on this activity/exam and that all work will be my own.